

```
$ riskwright --serve
```

Riskwright

An AI-powered credit risk platform. Predicts default probability, explains every decision, derives credit policy rules, and answers plain-English questions about the data.

Home Credit Default Risk | 307,511 applications | 8.07% default rate | 120 features
NeoStats AI Engineer assignment

```
$ docker-compose up
```

Architecture

All business logic in `src/`. The UI computes nothing.

```
React + nginx      no logic, HTTP client only
  |
  v /api/*
FastAPI             thin wrapper: validates, shapes
  |
+-----+-----+
  v           v           v
src/ml      src/data      src/talk_to_data
model, SHAP, loader,      NL to SQL,
rules, audits preprocess  validation, memory
  |           |           |
+-----+-----+
                v
              PostgreSQL
application_train, bureau, previous_application
```

FOUR SERVICES, ONE COMMAND

- > **postgres** – healthchecked
- > **loader** – one-shot, idempotent
- > **api** – waits on both
- > **frontend** – waits on api health

No flags, no override file. Startup races are the usual reason a submission does not run on someone else's machine, so the dependency conditions are load-bearing rather than decorative.

```
$ python -m src.ml.evaluate
```

From score to decision

Calibration first, then cost. The second is meaningless without the first.

1 - THE RAW OUTPUT IS NOT A PROBABILITY

Training with `scale_pos_weight = 11.39` multiplies the predicted odds by that weight. An average applicant scores near 0.5, not near 0.08.

2 - A KNOWN CONSTANT, SO INVERTED EXACTLY

```
p_true = p_w / (p_w + w(1 - p_w))
```

A weighted 0.5 maps back to $1/(1+w) = 0.0807$, the measured default rate. That identity is asserted in the test suite.

3 - ONLY NOW DOES A COST THRESHOLD MEAN ANYTHING

A false negative is a defaulted loan; a false positive is a good customer refused. Assumed **10:1**. Threshold minimises $10 \cdot \text{FN} + \text{FP}$.

THRESHOLD	FLAGGED	RECALL	PRECISION	COST
Naive 0.5	0.0%	0.002	0.659	247,681
Cost-optimal 0.0837	28.9%	0.644	0.180	161,314

Catching 64.4% of defaulters instead of 0.2%. Expected cost down 34.9%.

```
$ python -m src.ml.threshold_sensitivity
```

Risk bands, and what they rest on

The two boundaries answer different questions, and one of them is an assumption.

BOUNDARY	VALUE	HOW IT IS SET	COST RATIO	T_HIGH	APPROVE	CAUGHT
t_high Medium/High	0.0837	Cost-optimal	3:1	0.2284	94.9%	5,215
t_low Low/Medium	0.0341	Business judgment	4:1	0.1874	91.5%	7,465
			5:1	0.1615	88.6%	9,172
			6:1	0.1218	81.9%	12,316
			8:1	0.1068	78.4%	13,637
			10:1	0.0837	71.1%	15,979
			12:1	0.0689	64.8%	17,624
			15:1	0.0561	57.8%	19,182
			20:1	0.0407	46.4%	21,150

Above `t_high` the expected cost of approving exceeds the cost of refusing. Where automatic approval should *stop* is a matter of review capacity, which nothing in the data answers, so `t_low` is exposed as a policy dial rather than presented as a computed result.

HOW MUCH RESTS ON THE 10:1 ASSUMPTION

`t_high` moves 0.0407 to 0.2284 – a spread of 224% of the shipped value. Only 51.5% of applicants keep the same band across the range.

Every ratio still beats 0.5. The method survives where the number does not; what is uncertain is where to put the threshold, not whether 0.5 is wrong. It is.

```
$ python -m tests.run_chatbot_variance
```

Talk-to-data, measured like a model

44 held-out questions written after the prompt was frozen. Run five times.

38.4

MEAN OF 44

37-40

RANGE

36

STABLE AT 5/5

```
runs    39  37  40  39  37    out of 44
sd       1.34
fail    H35, H36  (every run)
```

Development set: 30/30. It was tuned on the failures it exposed, so its 100% is optimistic by construction.

THE SCORE IS A DISTRIBUTION, NOT A NUMBER

A single run once scored 19/20 and was reported as the result. It was the best of five. Quoting the good draw is how an evaluation flatters itself, which is why this is now run five times and reported as a spread.

TWO FAILURES ARE MINE, NOT THE MODEL'S

H35's reference counts ids in `bureau` where the model joins to `application_train` first – the model's reading is better. H36 asks it to exclude sentinel codes the schema never identifies. Reported, not repaired: fixing a reference after watching the model disagree is fitting the grader to the outcome.

```
$ src/talk_to_data/query_runner.py
```

Refusing is a feature

Two independent mechanisms: a hard gate, and a sanctioned way to decline.

STRUCTURED OUTPUT

One model call returns an action: `sql`, `refuse`, or `clarify`. A model with no sanctioned way to say "I cannot" invents a column to fill the silence.

VALIDATION GATE

Every generated statement is parsed, not string-matched. Single SELECT, whitelisted tables, columns checked against the live schema, LIMIT forced, 10s timeout, executed as a role holding SELECT and nothing else.

Rejected `WITH x AS (DELETE FROM application_train RETURNING sk_id_curr)`
 `SELECT count(*) FROM x`

Allowed `SELECT count(*) FROM application_train`
 `WHERE occupation_type = 'Delete'`

A DELETE hidden inside a CTE is rejected. The literal string 'Delete' is not. String matching gets both of these wrong; parsing the tree gets both right.

THE LIMIT OF IT

Querying a column that does not exist is impossible – a hard guarantee. Declining a question unanswerable *from columns that do* is a model judgement: over ten runs H8 declined seven times, and H44 answers a causal question as an association four times in five. 06 / 21

```
$ src/data/preprocessor.py
```

Fair Lending

Five protected attributes excluded. One of them got back in anyway.

EXCLUDED	PROTECTED BASIS
code_gender	Sex
name_family_status	Marital status
age_years	Age
cnt_children	Familial status
cnt_fam_members	Familial status

ECOA names sex, marital status and age as protected bases. A model that prices or refuses credit on them is a legal failure however well it performs.

FEATURE SET	ROC-AUC	PR-AUC
With protected	0.7651	0.2491
Without (shipped)	0.7614	0.2427

Cost of exclusion: 0.0037 ROC-AUC. There is no meaningful accuracy argument for keeping them.

EXCLUSION DID NOT HOLD

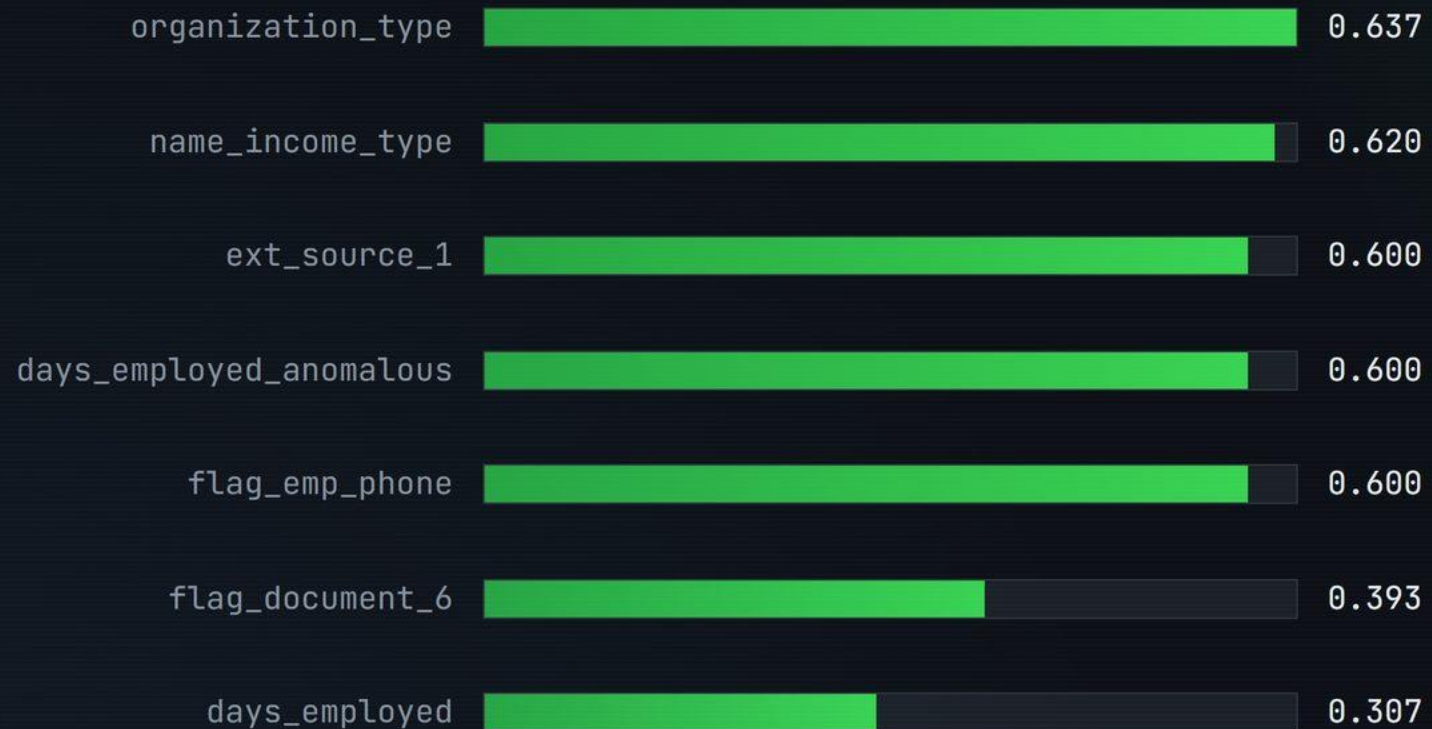
15 features are a material proxy for at least one excluded attribute. Age is the worst: 12 material, 5 strong.

```
$ python -m src.ml.proxy_detection
```

Systematic proxy detection

All 120 features scored against each excluded attribute.

FEATURES THAT RECONSTRUCT AGE, ALL 307,511 APPLICANTS



Thresholds fixed in the module before the first run: material at 0.20, strong at 0.50. A cutoff chosen after seeing the ranking describes the output rather than judging it.

NEITHER METHOD ALONE WAS ENOUGH

This scan would have **missed** `employed_life_ratio`: it predicts age at only 0.074, far below the floor, yet varying `days_birth` alone moves the score for 71.8% of applicants who have a `days_employed` value and **0.0%** of those who do not.

Hand-inspection found that one and none of the 12. Running both produced the full picture.

THE UNCOMFORTABLE ONE

`ext_source_1` reconstructs age at 0.600 and is the third most important feature in the model. Excluding age here does not exclude it from whatever the external bureau is scoring.

```
$ python -m src.ml.fairness_audit
```

Fair lending: disparate impact

Every applicant scored, threshold 0.083669, approval rate 70.39%.

AGE BAND – THE WORST-FAILING ATTRIBUTE

AGE BAND	N	APPROVED	OBSERVED DEFAULT
under 30	45,186	0.4873	0.1144
30-45	123,737	0.6807	0.0900
45-60	103,287	0.7698	0.0656
60+	35,301	0.8698	0.0492

ATTRIBUTE	4/5 RATIO	RESULT
code_gender	0.8528	PASS
age_band	0.5602	FAIL
name_family_status	0.7885	FAIL

Below 0.8: age_band, name_family_status

A SCREEN, NOT A VERDICT

Disparate impact in the legal sense requires a business-necessity analysis and a search for a less-discriminatory alternative. Neither is performed here. Observed default rates move with the approval rates, so the model tracks a real pattern through whatever it can reach – whether that justifies the disparity is exactly the analysis not done.

\$ POST /adverse-action

Adverse action reason codes

ECOA requires the specific principal reasons for a denial, not a score.

#	REASON GIVEN TO APPLICANT 100002	DRIVEN BY	SHAP
1	Credit assessment obtained from an external credit bureau	ext_source_1/2/3	0.847
2	Length of the repayment term relative to the amount requested	credit_term	0.221
3	Value of the goods financed relative to the credit requested	amt_goods_price	0.159
4	Length of time in your current employment	days_employed	0.088

WHY THE PHRASES ARE FIXED IN SOURCE

A regulated disclosure must produce the same words for the same input, and a notice that cannot be reviewed before it is sent cannot be signed off. Four maximum, per Regulation B. Duplicates collapse – three bureau scores are one reason to an applicant. An applicant below the threshold gets no notice, and the reasons are not computed rather than computed and hidden.

WHAT IT IS NOT

Still SHAP attributions shaped as reasons, not a rule-based engine working from the credit policy. Some features are never disclosed at all – appointment timing, the geographic columns, and the social-circle columns that describe other people's conduct – and the response names what it withheld rather than dropping it silently.

```
$ python -m src.ml.train
```

Model selection

Two models, identical inputs, 5-fold stratified cross-validation.

MODEL	ROC-AUC	PR-AUC
LightGBM (shipped)	0.7614 ± 0.0026	0.2441 ± 0.0008
Logistic regression	0.7438 ± 0.0026	0.2181 ± 0.0031

LightGBM wins by 0.0176 ROC-AUC – real, being several times the fold standard deviation, but less than a headline would suggest. A logistic regression gets within 2.4% of a gradient-boosted ensemble on this data.

That is a finding, not a disappointment. Logistic regression is what banks deploy under regulatory pressure, because a coefficient per feature is auditable in a way an ensemble is not.

WHY LIGHTGBM IS SERVED ANYWAY

- > SHAP **TreeExplainer** is exact and fast on trees
- > Rule derivation is naturally a shallow tree
- > NaN and categoricals handled natively, so no imputation

CLASS IMBALANCE

`scale_pos_weight` = 11.39, computed from each training split rather than hardcoded. SMOTE was not used: at this row count it is slow and addresses the same problem less directly. PR-AUC is reported alongside ROC-AUC everywhere, because on a target this imbalanced ROC-AUC alone flatters a model.

\$ notebooks/eda.ipynb

What the data says

Five insights, computed server-side and served from a single source.

307,511

APPLICATIONS

8.07%

DEFAULT RATE

11.4:1

IMBALANCE

122

COLUMNS, 67 WITH GAPS

THE DEFECT THAT MATTERED MOST

18.0% of applicants have `days_employed` set to 365243 – roughly 1,000 years. It encodes "not currently employed" rather than a duration. Left untreated it corrupts every statistic built on the column.

So what: those applicants default at 5.4% against 8.7% for the rest, so the flag is predictive and is kept as a feature rather than discarded.

FOUR MORE, SERVED FROM ONE PLACE

- > External credit scores separate risk more than anything self-reported
- > Education level tracks default risk strongly
- > Borrowing a lot relative to income does **not** predict default
- > Younger applicants default substantially more often

Every figure here is computed by the same module the API and the notebook call, so the deck, the UI and the analysis cannot quote different numbers for the same finding.

```
$ python -m src.ml.shap_full
```

Explainability

SHAP TreeExplainer. Exact on tree models rather than approximated.

GLOBAL IMPORTANCE – MEAN |SHAP|, ALL 307,511 APPLICANTS



EXHAUSTIVE, AND IT CHANGED NOTHING

Previously a 2,000-row sample, listed as a limitation. Recomputed over the whole population in chunks: the top 20 order is **identical** and the rank correlation across all 120 features is **0.9990**.

That is a finding, not a null result: the sample was adequate, and every SHAP figure previously reported was already correct.

PER PREDICTION

Endpoints return JSON, never a rendered image, so the UI owns presentation and the frontend stays replaceable. Each explanation carries a plain-English narrative generated deterministically from a label map – it cannot invent a reason the model did not use.

\$ GET /rules

Derived credit rules

A depth-3 surrogate short enough to sit in a policy document.

RULE	CONDITION	APPLICANTS	DEFAULT RATE	LIFT
R1	external credit score 2 is at most 0.460 and external credit score 3 is at...	5.5%	24.1%	2.98x
R2	external credit score 2 is above 0.460 and external credit score 3 is at m...	4.4%	15.6%	1.93x
R3	external credit score 2 is at most 0.460 and external credit score 3 is ab...	15.5%	13.7%	1.70x
R4	external credit score 2 is at most 0.248 and external credit score 3 is ab...	3.9%	9.9%	1.23x
R5	external credit score 2 is above 0.460 and external credit score 3 is abov...	5.4%	9.5%	1.18x
R6	external credit score 2 is above 0.460 and external credit score 3 is abov...	30.1%	5.9%	0.73x

TWO SETS, BECAUSE ONE IS NOT USABLE

The faithful set agrees with the model on **79.8%** of applicants but keys almost entirely off external bureau scores, so it says little a credit officer can act on. The policy set excludes those scores: 71.4% agreement, and rules that still apply to an applicant with no bureau history.

HONEST ABOUT WHAT THEY ARE

A depth-3 surrogate tree approximating the model's behaviour. These are a readable approximation for credit policy, **not the model itself**, and the gap is where a rule-based decision would differ from a scored one. Leaf statistics are counted from the data rather than read off the tree's internal values, which are reweighted.

\$ `pytest`

Engineering

Verified against a running stack, not asserted.

DOCKER

Four services. Postgres healthchecked; the loader waits for healthy and the API waits for the loader to exit successfully. Verified by cloning this repository into a fresh directory and building with `--no-cache`.

SECURITY

The chatbot connects as a role holding SELECT on three tables and nothing else. Verified: INSERT, CREATE and DROP are all refused by Postgres. The LLM key is scoped to the api service and never reaches the UI container.

176 TESTS, NO DATABASE NEEDED

The validation gate, the train/serve skew guard, the calibration identity, path resolution, four-fifths arithmetic, proxy statistics, adverse action disclosure, and the API contracts – one of which feeds a row carrying every protected attribute through the real serving path and asserts none reaches a SHAP contribution.

TOKEN COST

The schema block is 1,656 tokens, not the ~6,000 a full dump would be. Caching is wired but measured not to engage: Haiku 4.5 needs a 4,096-token prefix. Confirmed by padding to 7,393 tokens and watching the cache write, then read.

```
$ cat README.md | grep -A40 'Known limitations'
```

What this is not

The limitations that would matter to someone deciding whether to trust it.

FAIR LENDING IS INCOMPLETE

Two of three attributes fail the four-fifths screen and 15 features are material proxies for an excluded attribute. The screen, the proxy scan and the reason codes are built; the business-necessity analysis that would interpret them is not, nor is multivariate proxy detection or reject inference.

THE COST RATIO DECIDES HALF THE BOOK

10:1 is an assumption. It is now a bounded one, and the bound is wide: only 51.5% of applicants keep their band across 3:1 to 20:1.

THE MODEL IGNORES TWO OF THREE TABLES

`bureau` and `previous_application` serve the chatbot only. Prior credit history is the largest improvement available, worth roughly 0.02-0.03 ROC-AUC in published work.

THE EVALUATION IS SELF-AUTHORED

44 questions written by the person who wrote the prompt, and two of the failures are defects in those questions rather than model failures. An independently authored set is the next step.

NO AUTOMATED UI COVERAGE

The 176 tests cover `src/` and the `app/` contracts. The React UI is verified by hand. Conversation memory is in-process and dies on restart.

```
$ open http://localhost:5173
```

Data understanding

Dataset summary, the five insights, and the distribution explorer.

Riskwright

credit risk

Data understanding

Risk prediction

Why this decision

Derived rules

Ask the data

Database

connected

Model

loaded

Assistant

claude-haiku-4-5

Data understanding

Every figure below is computed by the API from Postgres. The same module backs the notebook, so the numbers here and in the analysis cannot drift apart.

Applications

307,511

Columns

122

Default rate

8.07%

Imbalance

11.4 to 1

24,825 defaulted, 282,686 repaid. 67 of 122 columns contain missing values.

What the data says

▼ A sentinel value hides in the employment column

18.0% of applicants have days_employed set to 365243, roughly 1,000 years, which encodes 'not currently employed' rather than a duration. Left untreated it corrupts every statistic built on the column.

So what: These applicants default at 5.4% against 8.7% for the rest, so the flag is predictive and is kept as a feature rather than discarded.

> External credit scores separate risk more than anything the applicant reports

> Education level tracks default risk strongly

> Borrowing a lot relative to income does not predict default, which is not what you would expect

> Younger applicants default substantially more often

Distribution explorer

Column

age_years

Bins: 30

17 / 21

```
$ open http://localhost:5173
```

Risk prediction

Score with band and recommended action.

Riskwright credit risk

Data understanding

Risk prediction

Why this decision

Derived rules

Ask the data

Databaseconnected

Modelloaded

Assistantclaude-haiku-4-5

Risk prediction

Probabilities are calibrated back from the imbalance weighting, so the number shown is a real probability of default rather than a ranking score.

An existing applicant

A what-if applicant

Applicant

100002

The first 60 applicants by id. These are training rows: the served model is refit on all of them, so a score here is a demonstration of the interface, not evidence of out-of-sample performance.

Age

26

Annual income

202,500

Loan amount

406,598

High

44.91%

Decline, or escalate for senior review

0%

Low below 3.41%

High at or above 8.37%

The upper boundary is the cost-optimal threshold, derived from a stated 10:1 cost ratio. The lower one is a business judgment about where automatic approval should stop, not an optimum.

18 / 21

```
$ open http://localhost:5173
```

Why this decision

SHAP contributions and the plain-English narrative.

Riskwright credit risk

Data understanding

Risk prediction

Why this decision

Derived rules

Ask the data

Databaseconnected

Modelloaded

Assistantclaude-haiku-4-5

Why this decision

SHAP values from a TreeExplainer, exact on tree models. The API returns them as JSON and this page draws them, so the explanation is not a picture rendered on the server.

High44.91%

This applicant has a 44.9% estimated chance of default, which places them in the High risk band. Risk is pushed up mainly by external credit score 3, external credit score 1 and external credit score 2. It is pulled down by flag phone, amt req credit bureau year and loan amount.

What moved this score

flag phone-0.028

amt req credit bureau year-0.027

loan amount-0.027

education level0.044

def 60 cnt social circle0.045

age of car owned0.048

occupation0.075

employer type0.076

length of employment0.088

def 30 cnt social circle0.144

price of the goods financed0.159

loan term0.221

external credit score 20.360

external credit score 10.489

external credit score 30.647

19 / 21

\$ open http://localhost:5173

Derived rules

Both rule sets with support, default rate and lift.

Riskwright

credit risk

Data understanding

Risk prediction

Why this decision

Derived rules

Ask the data

Database

connected

Model

loaded

Assistant

claude-haiku-4-5

Derived credit rules

A depth-3 surrogate tree fitted to the model's behaviour, so its logic is short enough to sit in a policy document. These approximate the model. They are not the model.

Faithful to the model

Policy usable

Agreement with the model

Base default rate

Rules

79.8%

8.07%

8

Rule	Condition	Applicants	Default rate	Lift
R1	external credit score 2 is at most 0.460 and external credit score 3 is at most 0.312	5.5% 17,005	24.1%	2.98x
R2	external credit score 2 is above 0.460 and external credit score 3 is at most 0.216	4.4% 13,457	15.6%	1.93x
R3	external credit score 2 is at most 0.460 and external credit score 3 is above 0.312 and external credit score 3 is at most 0.536	15.5% 47,551	13.7%	1.70x
R4	external credit score 2 is at most 0.248 and external credit score 3 is above 0.536	3.9% 11,908	9.9%	1.23x
R5	external credit score 2 is above 0.460 and external credit score 3 is above 0.216 and external credit score 3 is at most 0.327	5.4% 16,587	9.5%	1.18x
R6	external credit score 2 is above 0.460 and external credit score 3 is above 0.327 and external credit score 3 is at most 0.545	30.1% 92,426	5.9%	0.73x
R7	external credit score 2 is above 0.248 and external credit score 2 is at most 0.460 and external credit score 3 is above 0.536	7.5% 22,940	5.6%	0.70x
R8	external credit score 2 is above 0.460 and external credit score 3 is above 0.545	27.9%	3.1%	0.38x

20 / 21

```
$ open http://localhost:5173
```

Ask the data

Generated SQL is shown, not hidden.

📊 Data understanding

🔮 Risk prediction

🔍 Why this decision

🔗 Derived rules

📄 Ask the data

Databaseconnected

Modelloaded

Assistantclaude-haiku-4-5

Ask the data

Questions become SQL, which is parsed and checked against the live schema before it runs. Questions the data cannot answer are declined rather than guessed at, and the generated SQL is always shown.

You

What is the overall default rate?

Riskwright

The overall default rate is 8.07%.

default_rate

0.0807

Generated SQL

SELECT AVG(CAST(target AS DOUBLE PRECISION)) AS default_rate FROM application_train LIMIT 200

You

What is the average credit score of applicants?

Riskwright

🚫 Cannot answer this

There is no column called "credit_score" in the database. The dataset contains external credit bureau scores in columns ext_source_1, ext_source_2, and ext_source_3, which are normalized scores between 0 and 1 from external data sources.

Try instead: What is the average value of ext_source_1 (or ext_source_2 or ext_source_3) across all applicants?

How does default rate vary by education level?

⬆

Session k2y52db9. Conversation memory is held by the API, not by this page.

Clear conversation